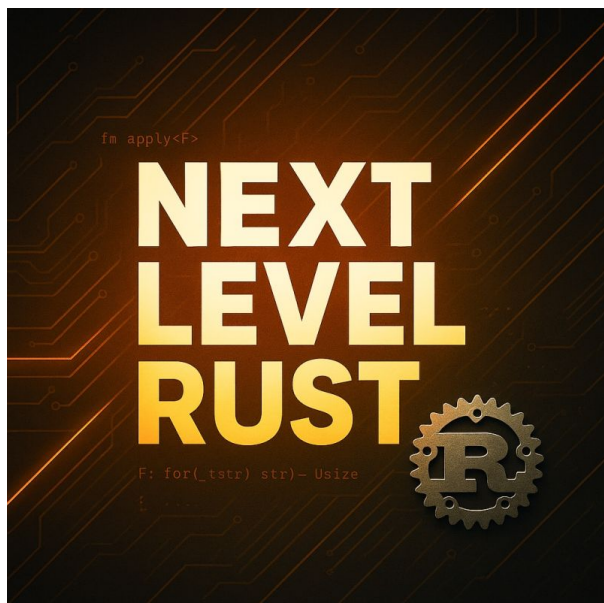




Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

Advanced cargo usage

Workspace-Level Configuration and Optimization

Workspaces allow you to manage multiple related crates as a single unit. This is especially useful for large projects, libraries with associated binaries, or when developing a suite of tools that share code. In this chapter, we explore how to configure and optimize a Rust workspace using only Cargo, without external tools.

What Is a Workspace?

A workspace is a collection of crates that share a **Cargo.lock** file and build directory. One of the crates is the root crate, and others are members. Workspaces help reduce duplication, speed up compilation, and simplify dependency management.

All crates in a workspace must reside in the same directory hierarchy, and the workspace is defined by a root **Cargo.toml** file.

Creating a Workspace

Let's create a workspace called **cli-toolkit** that includes a library and two binaries: a CLI frontend and a testing utility.

Step 1: Initialize the project

```
mkdir cli-toolkit
cd cli-toolkit
cargo new --lib library
cargo new --bin frontend
cargo new --bin tester
```

This creates a directory structure:

```
cli-toolkit/
├── Cargo.toml
├── library/
│   ├── src/lib.rs
│   └── Cargo.toml
├── frontend/
│   ├── src/main.rs
│   └── Cargo.toml
└── tester/
    ├── src/main.rs
    └── Cargo.toml
```

Step 2: Define the workspace

Edit the root `Cargo.toml` (in `cli-toolkit/`) to define the workspace:

```
[workspace]
members = [
    "library",
    "frontend",
    "tester",
]
```

This tells Cargo that these three crates belong to the same workspace.

Now, modify each member's `Cargo.toml` to ensure they don't define their own workspace settings.

For example, in `library/Cargo.toml`, add:

```
[package]
name = "library"
version = "0.1.0"
edition = "2021"

# No [workspace] section needed
```

In `frontend/Cargo.toml`, add a dependency on the local library:

```
[package]
name = "frontend"
version = "0.1.0"
edition = "2021"

[dependencies]
library = { path = "../library" }
```

Do the same for `tester/Cargo.toml`:

```
[dependencies]
library = { path = "../library" }
```

Building and Testing the Workspace

From the workspace root, you can build all crates:

```
cargo build
```

Or build just one:

```
cargo build -p frontend
```

Run tests across all crates:

```
cargo test
```

This runs all unit and integration tests in every member crate.

You can also run tests for a specific crate:

```
cargo test -p library
```

Shared Dependencies

You can define common dependencies at the workspace level if using a virtual workspace manifest. To do this, keep the root `Cargo.toml` without a `[package]` section.

Update the root `Cargo.toml`:

```
[workspace]
members = [
    "library",
    "frontend",
    "tester",
]

[workspace.dependencies]
clap = { version = "4.0", features = ["derive"] }
serde = { version = "1.0", features = ["derive"] }
```

Now, any crate in the workspace can use these dependencies without repeating the version.

In `frontend/Cargo.toml`, use:

```
[dependencies]
library = { path = "../library" }
clap = { workspace = true }
```

This ensures version consistency and reduces duplication.

Optimization: Avoiding Redundant Builds

Cargo builds all dependencies shared across workspace members only once, reusing artifacts. This speeds up compilation.

To further optimize, enable incremental compilation and choose an appropriate optimization level.

In `.cargo/config.toml` at the workspace root:

```
[build]
incremental = true

[profile.dev]
opt-level = 1
```

This improves debug build performance.

For release builds, use:

```
[profile.release]
opt-level = "z"
```

```
lto = true
codegen-units = 1
```

These settings reduce binary size and improve performance.

Create the file `.cargo/config.toml` in the workspace root to apply these settings to all members.

When a `Cargo.toml` like this is placed inside the `.cargo` folder of a workspace, it applies **global build settings** to all crates in the workspace.

- **[build] incremental = true** Enables incremental compilation across the whole workspace. This makes rebuilds faster because Cargo reuses previously compiled code.
- **[profile.dev] opt-level = 1** Development builds (default `cargo build`) will use a small amount of optimization instead of none. This makes binaries a bit faster while keeping compile times reasonable.
- **[profile.release]** These settings affect optimized builds (`cargo build --release`) for all crates:
 - **opt-level = "z"** → prioritize smallest possible binary size.
 - **lto = true** → enable Link-Time Optimization, letting the compiler optimize across crate boundaries.
 - **codegen-units = 1** → compile everything in a single unit, which increases optimization opportunities but makes compilation slower.

In short, this file ensures the entire workspace uses **faster incremental dev builds** and **highly optimized, size-focused release builds**.

Example Code

In `library/src/lib.rs`:

```
pub fn process_input(text: &str) -> String {
    text.to_uppercase().trim().to_string()
}
```

In `frontend/src/main.rs`:

```
use library::process_input;

fn main() {
    let input = "  hello world  ";
    println!("Processed: {}", process_input(input));
}
```

In `tester/src/main.rs`:

```
use library::process_input;

fn main() {
    assert_eq!(process_input(" rust "), "RUST");
    println!("Test passed!");
}
```

Run the frontend:

```
cargo run -p frontend
```

Output:

```
Processed: RUST
```

Run the tester:

```
cargo run -p tester
```

Output:

```
Test passed!
```

Cleaning and Managing Output

To clean build artifacts for the entire workspace:

```
cargo clean
```

To update dependencies:

```
cargo update
```

This updates the shared `Cargo.lock` file.

Summary

Workspaces simplify managing multiple crates by enabling shared configuration, consistent dependencies, and efficient builds. By using a root `Cargo.toml` with `[workspace]` and `[workspace.dependencies]`, you reduce redundancy and improve maintainability.

Optimize your workspace with profile settings and shared build configurations. All of this is achievable using only Cargo commands and standard Rust tooling.

With workspaces, you can scale your projects cleanly from a simple library and binary pair to a full ecosystem of interdependent crates without complexity or external tools.